GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

Turbo：Efficiently Serving Long-Context LLMs with In-Network Aggregation

Ying Wan, Yuchen Xu, Chuwen Zhang, Yingsheng Huang,

Yong Feng, Wenquan Xu, Jialin Li, Mingwei Xu, Wenfei Wu,

Congcong Miao.



CONTENTS

Part 01 研究背景与关键问题

Part 02 Turbo 系统设计

Part 03 分析与实验评估

Part 04 总结



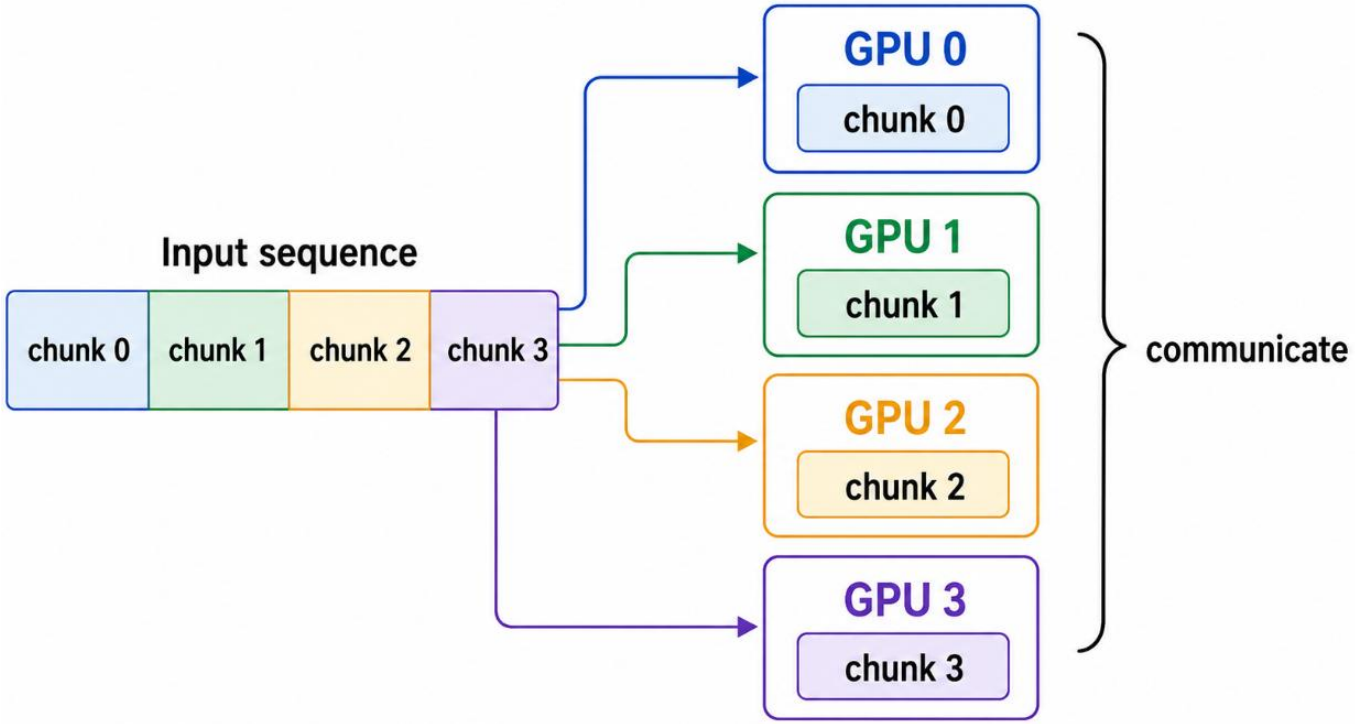
01

研究背景与关键问题



背景

- 长上下文的广泛应用：
  - RAG检索增强生成
  - 长文档理解
  - 多轮对话记忆
- 长上下文放大**开销**：
  - KV cache显存占用
  - 注意力计算量
  - GPU间通信量

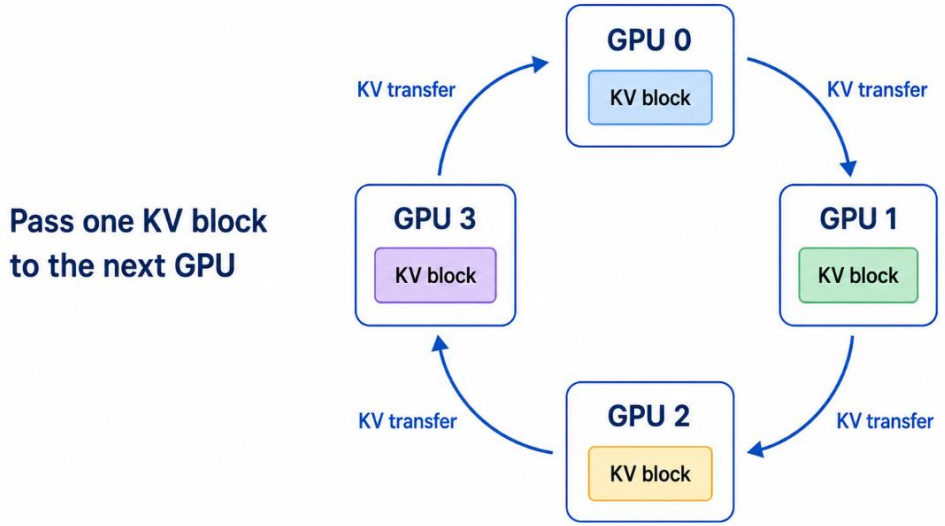


上下文并行：把输入序列拆分到多个GPU上，通过通信共享KV cache

现有方案&其在解码阶段的局限性

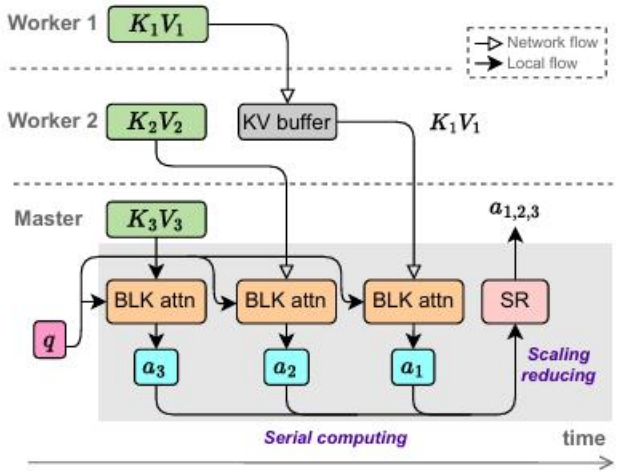
Pass-KV

- Ring： KV块在各工作节点间轮流传递，并最终汇总至主节点
- 局限性:
  - 通信数据量大
  - 解码延迟随工作节点数量的增加而增加

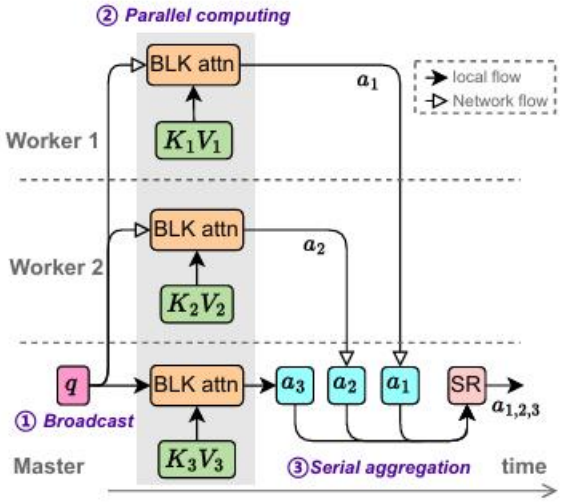


Pass-Q

- P2p：主节点广播q，并汇总各工作节点返回的部分注意力计算结果
- 局限性:
  - 减少了链路流量，但带来了 incast 汇聚拥塞和聚合阶段瓶颈



(a) Pass-KV (ring)



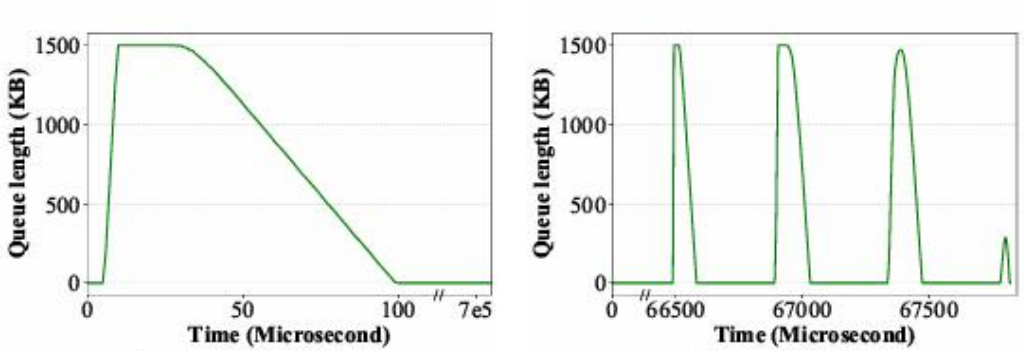
(b) Pass-Q (p2p)

- 集中式通信会加剧排队延迟和尾延迟

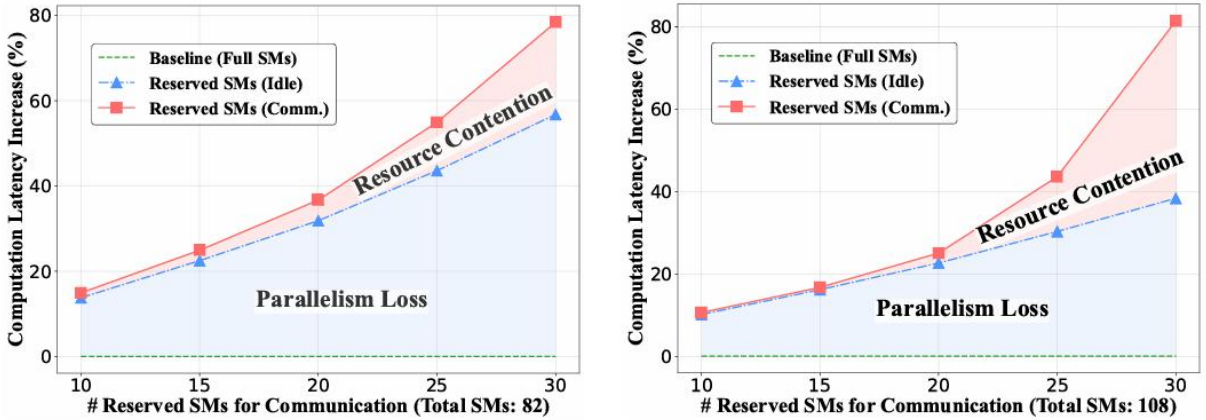
如何减少网络流量并避免 incast?

- 通信内核会占用GPU的流式多处理器（SM），从而降低矩阵计算性能

如何减少通信连接数来降低计算干扰?



(a) Pass-KV (p2p)　(b) Pass-Q (p2p)



(a) RTX3090 with PCIe　(b) A100 with NVLink

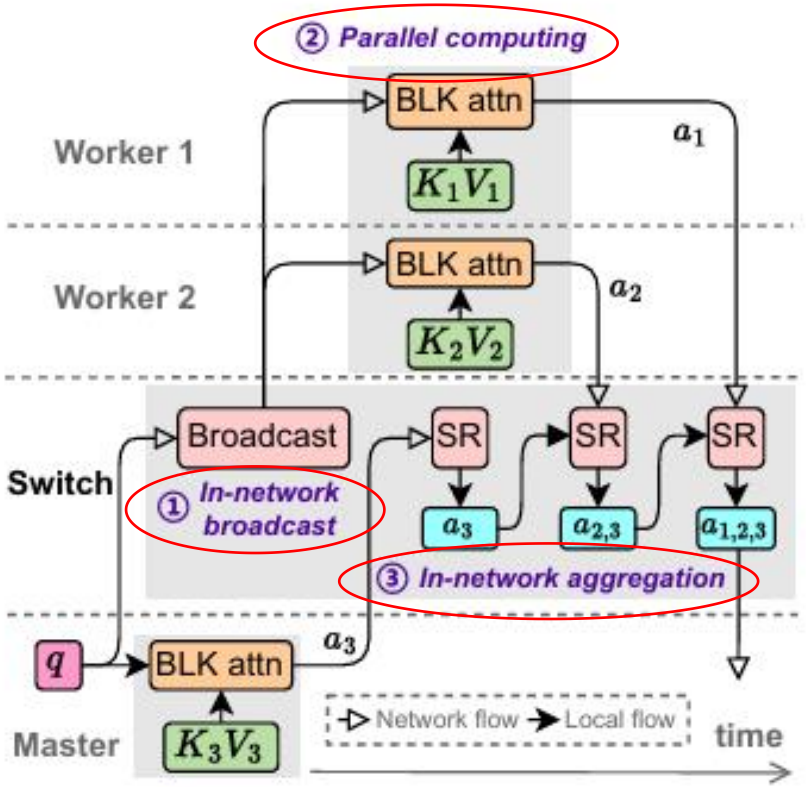
绿色: 所有SM都用于计算
蓝色: 将部分SM用于通信
红色: 将用于通信的SM与计算SM并发执行

02

Turbo 系统设计



- 解码过程更适合于 Pass-Q 范式
  - 广播查询向量
  - 并行计算注意力
- 将**查询广播**和**注意力聚合**卸载到可编程交换机上
  - 支持**并行计算**注意力
  - 主节点减少了通信/规约内核的开销
  - 主节点只需要接收最终的注意力结果
  - 保持**线速**转发



Turbo (ours)

- 控制平面
  - 调度序列
  - 配置交换机
  - 管理 KV cache
- 主节点
  - 负责发起解码请求
  - 发送查询并接收聚合后的注意力结果
  - **可与张量并行（TP）、数据并行（DP）等策略组合**
- 工作节点
  - 存储本地 KV 缓存分片
  - 接收查询向量，计算并返回部分注意力结果
- 交换机
  - 广播查询向量
  - 在线聚合部分注意力结果

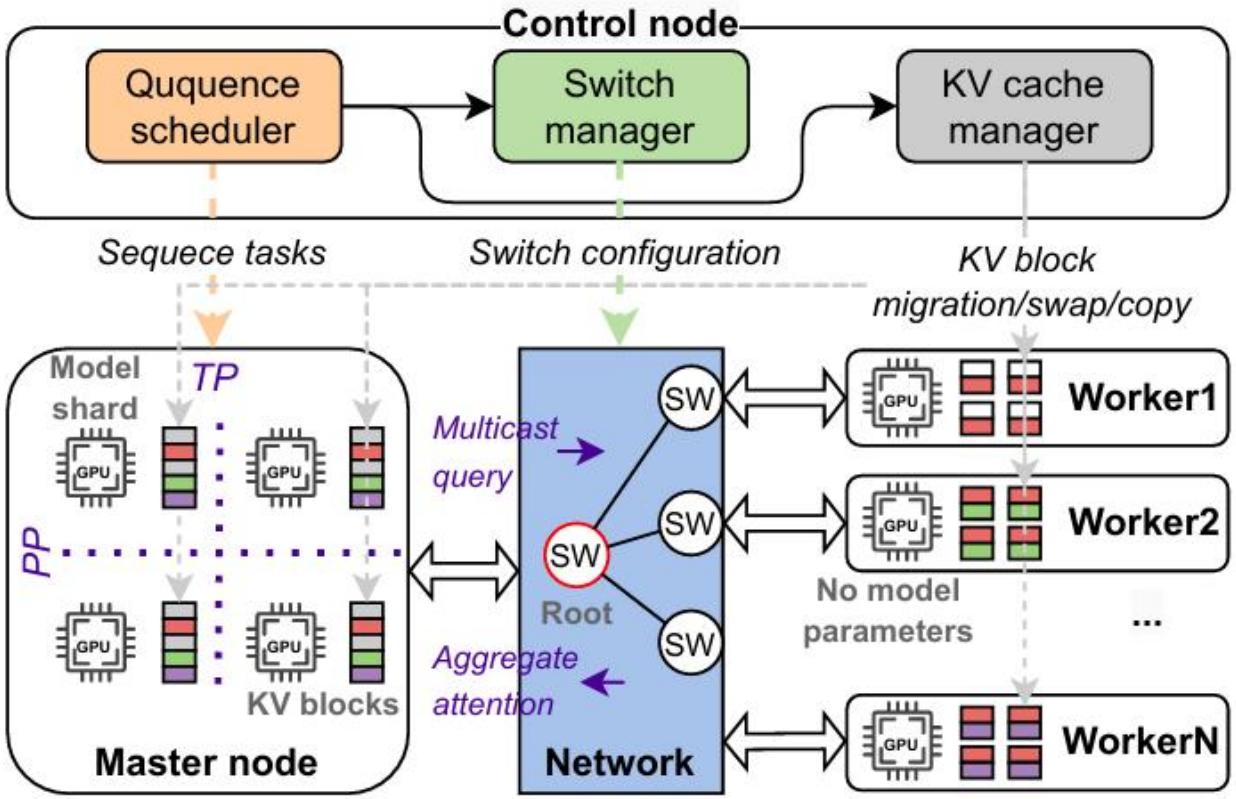
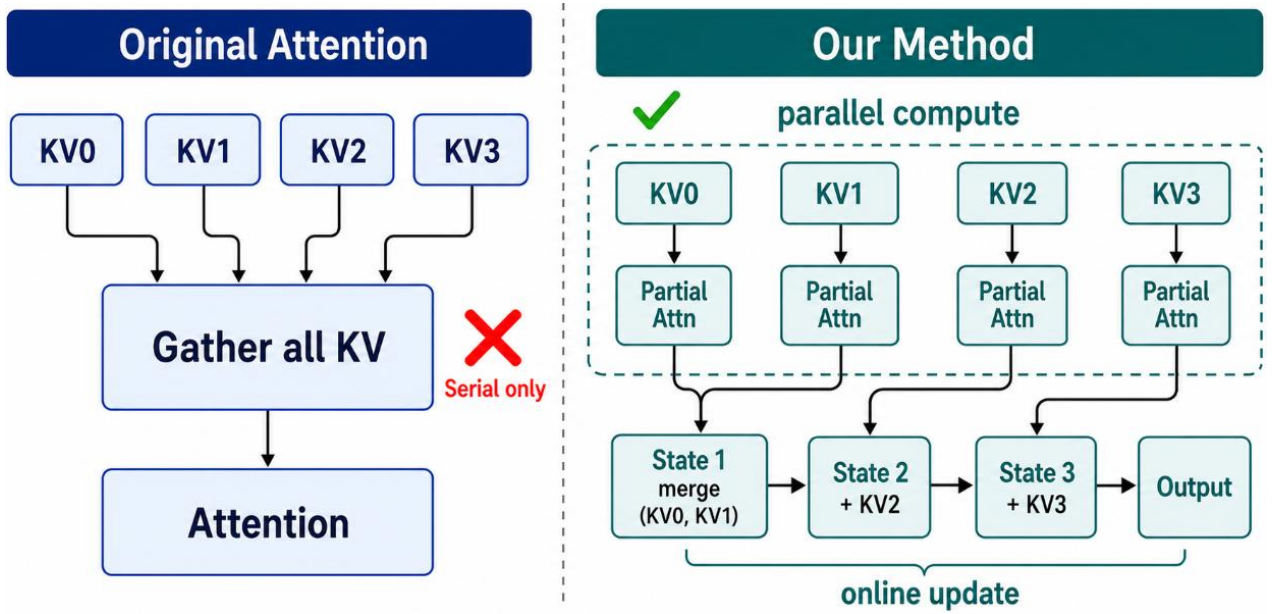
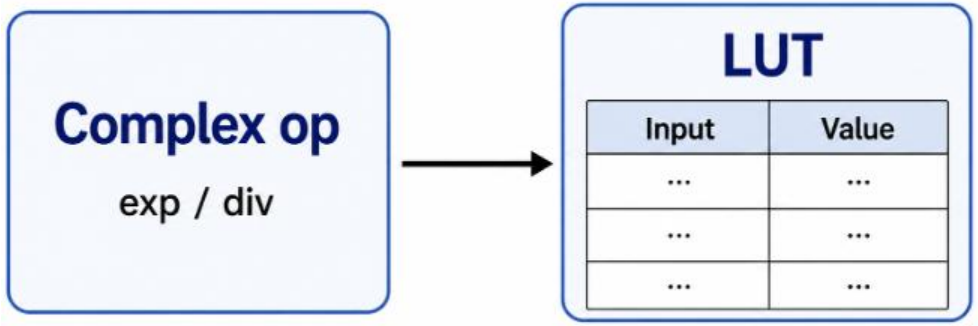


Figure 19: The inference architecture with Turbo.

挑战 1 | 在资源受限的交换机上实现注意力聚合

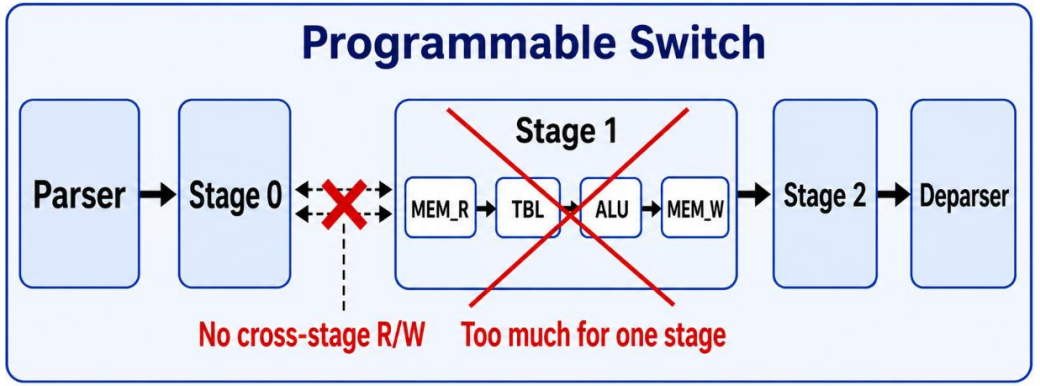
- 标准注意力聚合依赖于 softmax 归一化
  - 需要一次性**全局**规约
  - 使用复杂的**指数/除法等**运算
- 本方案设计
  - 将全局规约重写为**在线更新**
  - 使用**查找表**近似指数/除法运算
- 特点
  - 只需维护一个缓存区
  - 容忍部分注意力结果乱序到达

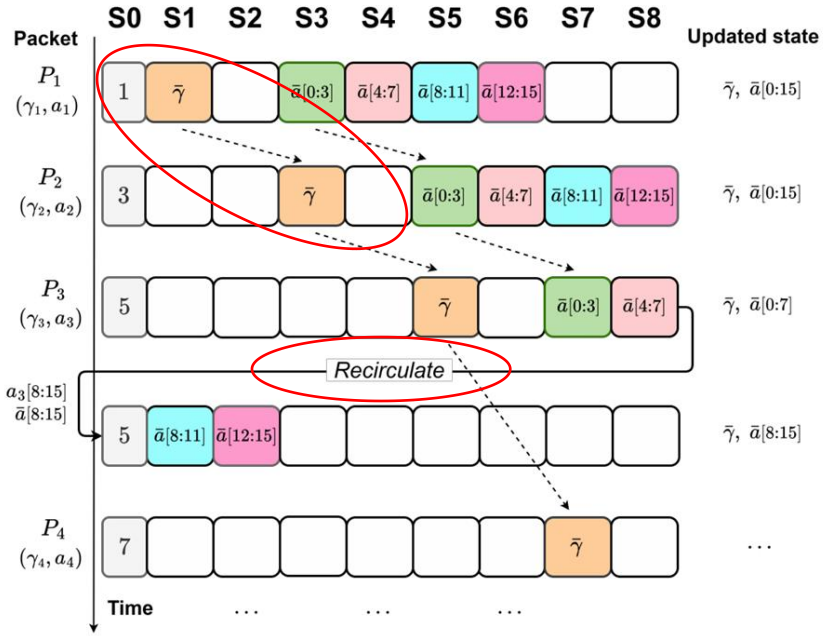




挑战 2 | 在RMT架构内实现跨阶段状态更新

- RMT 流水线架构的限制
  - 完整更新逻辑无法容纳在**单个阶段**中
  - **不允许**跨阶段读写状态
- 前向滚动状态更新机制
  - 将更新拆分到多个流水线阶段
  - 连续到达的数据包偏移两个阶段
  - 在正确的阶段读取前一状态
- 代价
  - 未完成的聚合需要重新循环





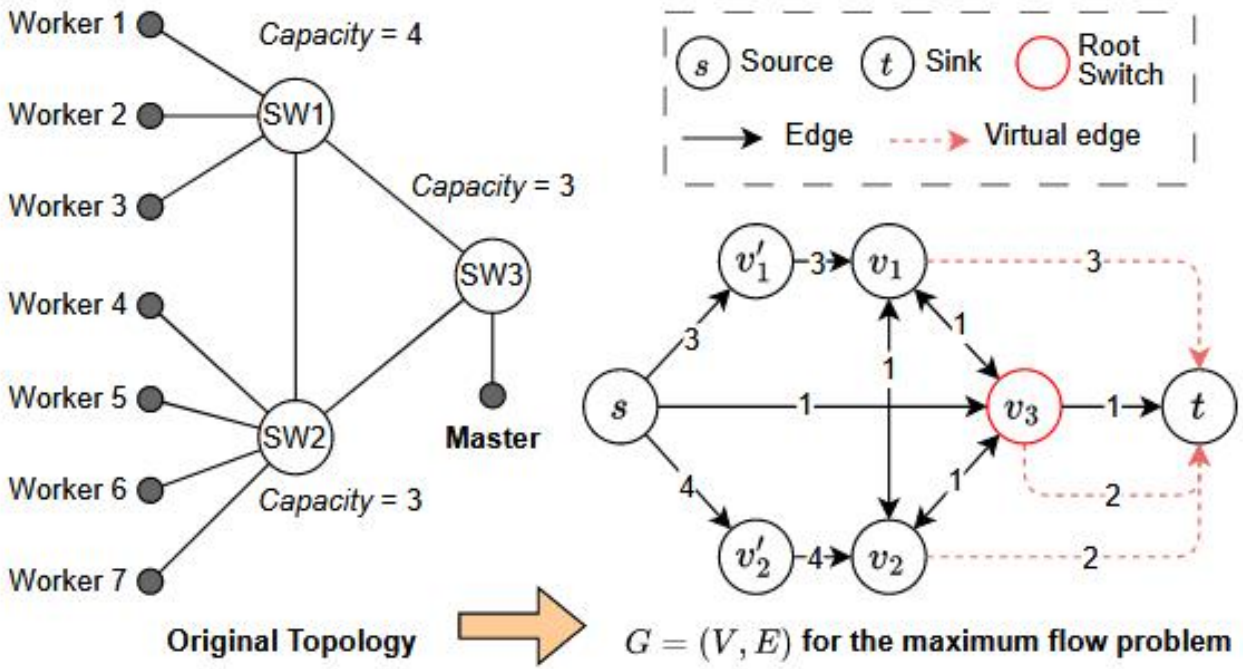
An example of the rolling forward state update.

- 避免让单个交换机**过载**
- 聚合树的构造

→ 线性规划建模

→ 最大流分配问题

- 将任务分配问题转化为一个容量受限的流问题
  - *Ford-Fulkerson算法*



$$
\begin{aligned}
\max \quad & \sum_{\forall u \in V_{\mathrm{wk}}} \sum_{e \in out(u)} f(e), \\
\text{s.t.} \quad & f(e) \le w(e), \ \forall e \in E \\
& \sum_{e \in out(u)} f(e) \le 1, \ \forall u \in V_{\mathrm{wk}} \\
& \sum_{e \in in(u)} f(e) - \sum_{e' \in out(u)} f(e') + 1 \le C(u), \forall u \in V_{\mathrm{sw}}.
\end{aligned}
$$

03

分析与实验评估



s： 序列长度, c ： KV 块大小, n： 工作节点数量（或KV块数量）， d： 隐藏维度

方案	网络流量	延迟	计算量	缓冲区大小
Pass-KV (p2p)	O(sd)	$T_{net}+n\times T_{block_attn}+T_{reduce}$	$n\times C_{block_attn}+C_{reduce}$	O(sd)
Pass-KV(ring)	O(sd)	$n \times \max\{T_{net,}\ T_{block_attn}\} + T_{reduce}$	$n\times C_{block_attn}+C_{reduce}$	O(cd)
Pass-Q (p2p)	O(nd)	$2T_{net}+T_{block_attn}+T_{reduce}$	$C_{block_attn}+C_{reduce}$	O(nd)
Pass-Q (ring)	O(nd)	$n\times\max\{T_{net}, T_{block_attn}\}+T_{reduce}$	$C_{block_attn}+C_{reduce}$	O(d)
Turbo	**O(d)**	**$2T_{net}+T_{block_attn}$**	**C_{block_attn}**	O(d)

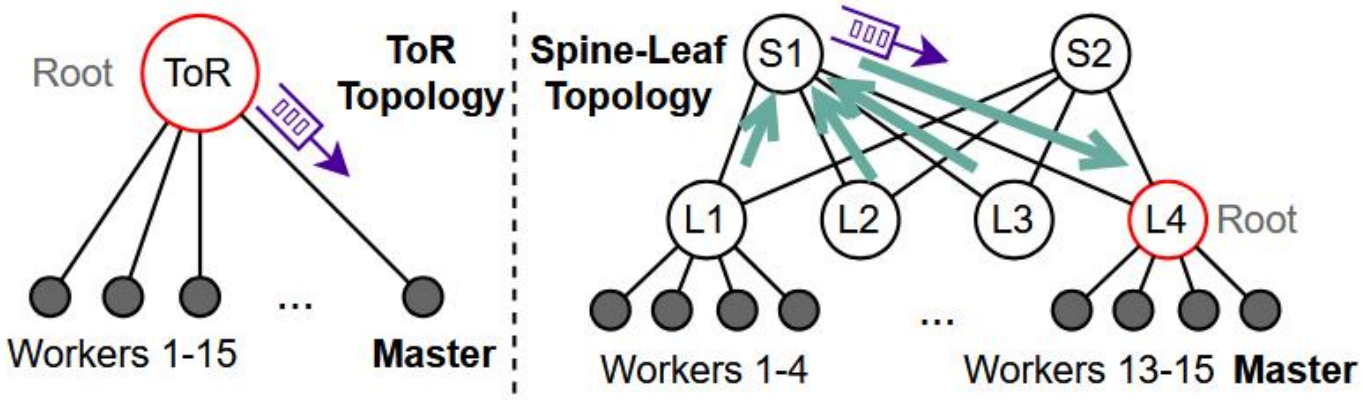
降低网络流量、端到端延迟、GPU计算开销和缓冲区需求

真实测试平台

- 4个节点× 2 张RTX 3090 GPU
- 100 Gbps NIC
- Tofino2 可编程交换机
- LLAMA-2 7B + ShareGPT

大规模评估

- 16 个节点, 200 Gbps RoCEv2
- ToR 和 Spine-Leaf 网络拓扑
- LLAMA-2 7B + GPT-3 175B
- 最多32 个并发任务



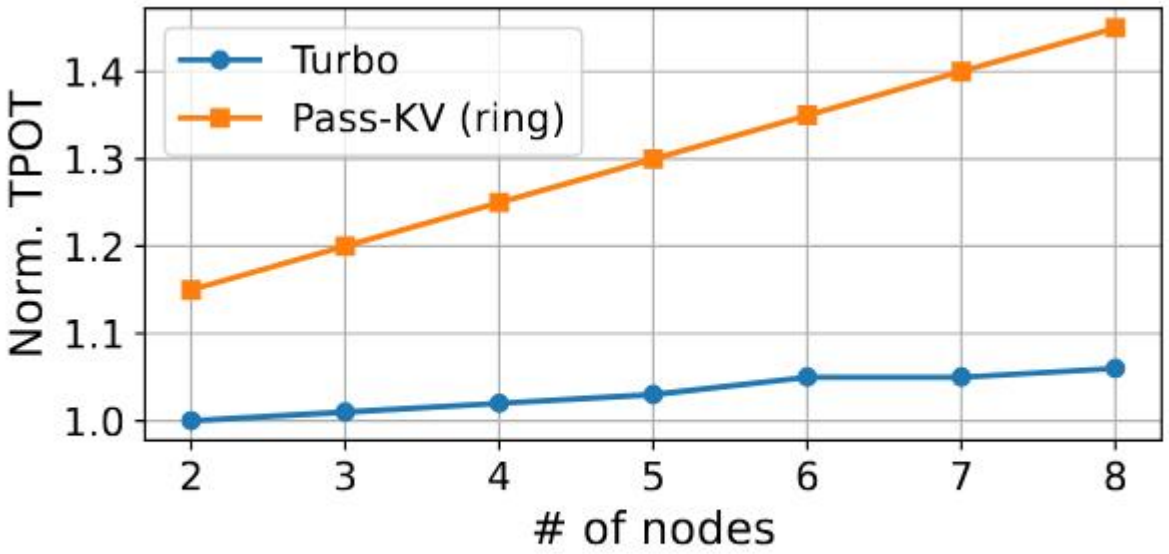
The two typical topologies used in the testbed.

TPOT：输出每个令牌所需的时间

解码延迟

8节点规模下对比 Pass-KV Ring方案

最高降低37%



End-to-end TPOT on our testbed.

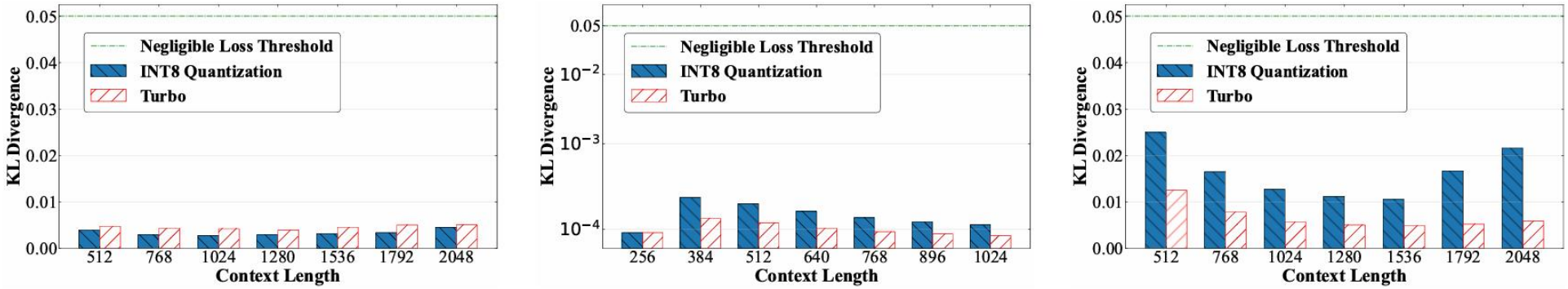
Turbo在节点数增多时展现出更优的扩展性能

准确性

- **困惑度**:衡量下一个token的预测质量
- **KL散度**:衡量相对于原始分布的偏移情况
- 两者都是越低越好

	FP16	Turbo	INT8 Quantization
LLAMA-2	5.5654	5.6966	5.6302
GPT-2	19.137	19.3537	19.4281
Qwen-2.5	6.9435	7.462	7.8577

Perplexity comparison across three LLMs.



(a) LLAMA-2 (b) GPT-2 (c) Qwen-2.5

End-to-end inference distribution fidelity (KL Divergence) across different LLM models.

Qwen 3.5 9B

上下文长度	50k	100k	150k	200k	250k
KL 散度	0. 007927	0. 009656	0. 008938	0. 009187	0. 008893

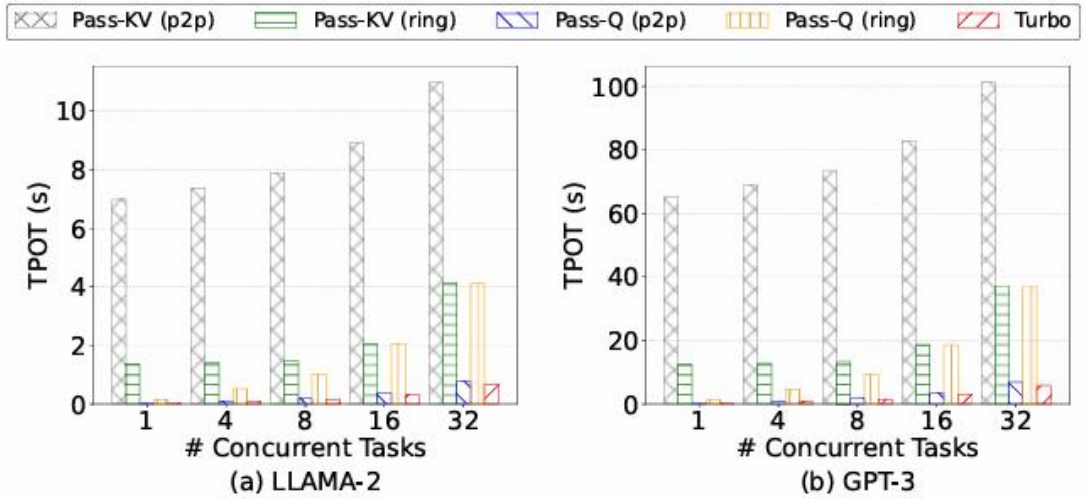
Turbo在近似误差可忽略不计的情况下，保持了输出质量。

大规模评估实验结果

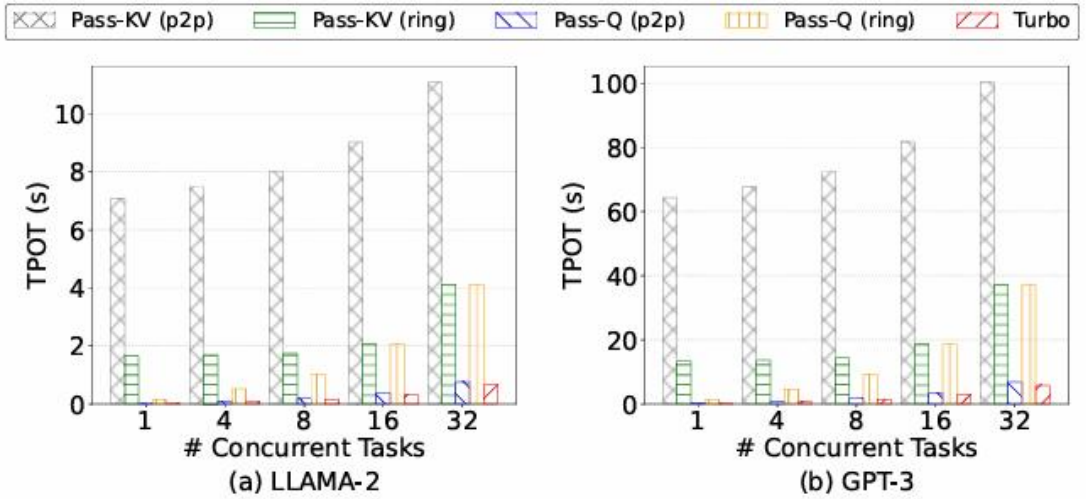
Turbo在降低主节点通信量的同时，保持了较低的TPOT

	Traffic size		Incast
	LLAMA-2	GPT-3	
Pass-KV (p2p)	30 GB	90 GB	✓
Pass-KV (ring)	30 GB	90 GB	✗
Pass-Q (p2p)	3.75 MB	11.25MB	✓
Pass-Q (ring)	56.25 MB	168.75MB	✗
Turbo	0.256 MB	0.768 MB	✗

Master-side inbound traffic volume.



TPOT on the ToR topology.



TPOT on the Spine-Leaf topology.

	LUT	FF	Block RAM
Turbo	40,768	67,072	128/1,024
All resources	1,303,680	2,607,360	2,016
Ratio (%)	3.13	2.57	6.3/50.29

Resource consumption of Turbo on FPGA U280.

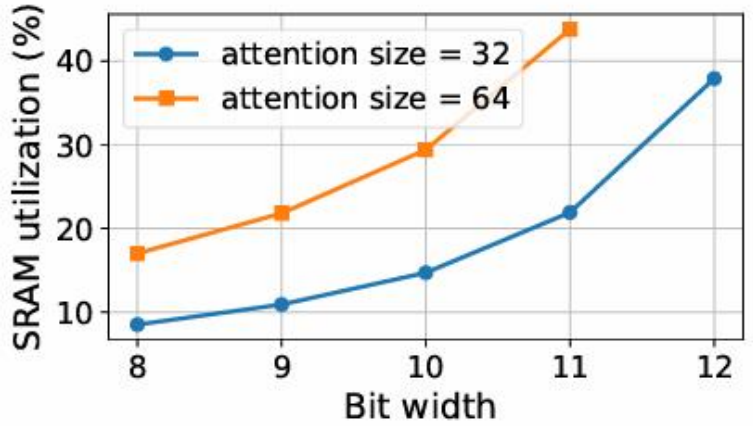


Figure 13: SRAM consumption on Tofino2.

ᶜRAM consumption on Tofino2.

Turbo的额外硬件资源开销是可接受的

04

总结



总结

- 关键观察:
  - 长上下文解码暴露了 CP 中的通信与聚合瓶颈
- 核心思想:
  - 将查询向量广播和注意力聚合卸载到可编程交换机上执行
- 贡献:
  - 提出了在线查找表注意力聚合机制、前向滚动状态更新机制，以及负载感知的聚合树
- 实验结果:
  - 更低解码延迟、显著减少通信流量，同时只引入可接受的额外开销，并且精度损失可忽略

GOSIM

GOSIM Shenzhen 2026

Thanks!

